Minho University
Engineering School

Dong Xuyong

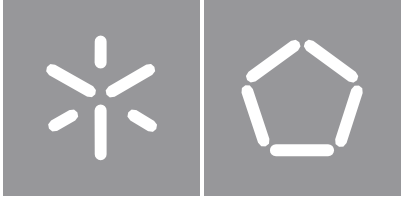
Hierarchical structure optimization for anomaly detection grouped time series data

February 2025

Hierarchical structure for anomaly detection in multi-dimensional time

Dong Xuyong

UMinho | 2024



Minho University
Engineering School

Dong Xuyong

Hierarchical structure for anomaly detection in multi-dimensional time series data

Master's dissertation in Information Systems Engineering and Management

Work carried out under the supervision of
Professor Doctor Paulo Alexandre Ribeiro Cortez
Professor Doctor Emanuel Demétrio Mendes Gouveia

February 2025

Acknowledgements

Summary

The master's thesis focuses on method anomaly detection in multidimensional telecommunications time-series data in collaboration with DTx. It proposes an agnostic method for discovering hierarchical aggregation structures and applying them to anomaly detection in complex grouped data. The proposal investigates the case of full or partial hierarchical structures rather than global aggregation or complete disaggregation for monitoring. At meaningful levels of detail, they can reveal abnormal patterns in traffic.

The thesis researches how evolutionary search can aid the development of more flexible and interpretable anomaly detection in telecommunications network monitoring. This is done specifically for network monitoring data with hierarchical attributes. Structures within a candidate hierarchy are created and assessed based on their ability to determine normality versus abnormality. The approach utilizes data aggregation at different scales, the generation of synthetic anomalies, Matrix Profile-based anomaly scoring, and the use of PonyGE2 for grammatical evolution.

Abstract

This master's proposal focuses on anomaly detection in multidimensional telecommunications time-series data using DTx. Modern telecommunications networks generate large volumes of traffic data that need to be monitored to detect anomalous behavior that can impact service reliability, capacity provisioning, and operational decision-making. Nonetheless, each of these datasets is generally structured along various categorical dimensions, creating a multitude of possible aggregation levels, thus making it difficult to decide which time series to analyze. The author proposes an agnostic framework for anomaly detection based on hierarchical aggregation structures. The authors do not support either a single global view or fully disaggregated monitoring.

The author instead considers exploring partial hierarchical structures that could reveal abnormal traffic patterns at a relevant level of detail. In the end, this approach will help detect anomalies in complex grouped datasets without sacrificing interpretability or incurring high analytical costs. A formal grammar automatically generates hierarchical structures, which are evaluated in terms of their ability to classify behavior as normal or anomalous. ~~In addition to predefined hierarchies~~, our method can automatically discover useful aggregation structures. To accomplish this, Grammatical Evolution (*Grammatical Evolution* (GE)) (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003), a nature-inspired computing concept, has been used. The major contribution of this thesis is a reproducible, interpretable framework for learning hierarchical anomaly-detection rules for telecommunications time series. It shows how to use evolutionary search and time-series anomaly scoring to learn useful rules despite the dimensionality of the network data. Telecom companies are facing increasing security threats to their networks, presumably due to the malicious intent of hackers, crackers, and others.

Keywords: anomaly detection, telecommunications, hierarchical time series, grouped time series, Grammatical Evolution, Matrix Profile.

Contents

Acknowledgements . . . 2
Summary . . . 3
Abstract . . . 4
List of Figures . . . 7
List of Tables . . . 8
List of abbreviations and acronyms . . . 9

1 Introduction **10**
1.1 Motivation . . . 10
1.2 Objectives and Expected Results . . . 10
1.3 Document Organization . . . 11
1.4 Methodological Approach . . . 11
1.4.1 Search Method . . . 11

2 Theory and Background **13**
2.1 Time Series . . . 13
2.1.1 Hierarchical Time Series . . . 13
2.1.2 Grouped Time Series . . . 14
2.2 Anomaly Detection . . . 15
2.3 Anomaly Generation . . . 15
2.3.1 Rule-Based Approaches . . . 16
2.3.2 Validation of Synthetic Anomalies . . . 16
2.4 Grammatical Evolution . . . 16
2.4.1 Combinatorial Optimization . . . 16
2.4.2 Genetic Programming . . . 17
2.4.3 Backus-Naur Form . . . 18
2.4.4 Initialization Phase . . . 20
2.4.5 Selection Mechanism . . . 21
2.4.6 Variation Operators . . . 21
2.4.7 Replacement Strategy . . . 21
2.5 Metrics . . . 22
2.5.1 Precision and Recall . . . 22
2.5.2 Trade-off and Applications . . . 22
2.5.3 Area Under the ROC Curve (AUC) . . . 23

3 Literature Review **24**
3.1 Anomaly Detection in Telecommunications Networks . . . 24
3.2 Anomaly Detection in Grouped Time Series . . . 24
3.3 Anomaly Detection in Hierarchical Time Series . . . 25

3.4 Matrix Profile and Time-Series Discord Detection 26
3.5 Combinatorial Optimisation for Selecting Time Series 26

4 Materials and Methodology **28**
4.1 Problem Formulation 28
4.1.1 Key Observations 29
4.1.2 Analytical Implications 30
4.2 Adopted Methodology 30
4.3 Model Development and Evaluation 33
4.3.1 Genome Structure 33
4.4 Anomaly Injection 34
4.5 Hierarchical Data Structures 35
4.6 Genetic Programming Implementation 35

5 Results and Discussion **36**
5.1 Performance Analysis 36
5.2 Time Series Visualization 36
5.3 Genetic Programming Analysis 36

6 Conclusions and Future Works **37**
6.1 Conclusions 37
6.2 Future Work 37

List of Figures

2.1 A two level hierarchical tree diagram 14
2.2 Australian domestic overnight trips over the period 1998 Q1 to 2017 Q4 disagrgreagted by purpose of travel and by state. 15
2.3 Generational loop in an evolutionary algorithm. 17

4.1 Vodafone network traffic metrics (#Events and Event Duration) over July-October 2022 aggregated. 28
4.2 Daily #Events from 18–28 July 2022 29
4.3 Anomaly detection workflow 31
4.4 PonyGE2 control flow diagram. 33
4.5 Anomaly detection workflow 34

List of Tables

4.1 Cardinality and Dominant Categories in Vodafone Data 29
4.2 Information Content by Variable 30
4.3 Key Configuration Parameters 35

List of abbreviations and acronyms

AUC *Area Under the Curve*

AUROC *Area Under the Receiver Operating Characteristic Curve*

BNF *Backus–Naur Form*

CDR *Call Detail Record*

CNN *Convolutional Neural Network*

DTW *Dynamic Time Warping*

GA *Genetic Algorithm*

GE *Grammatical Evolution*

GP *Genetic Programming*

ISP *Internet Service Provider*

KPI *Key Performance Indicator*

LOF *Local Outlier Factor*

LSTM *Long Short-Term Memory*

MAE *Mean Absolute Error*

MAP *Mean Average Precision*

MP *Matrix Profile*

ROC *Receiver Operating Characteristic*

ROI *Region of Interest*

VAE *Variational Autoencoder*

Chapter 1

Introduction

1.1 Motivation

Telecom networks produce many KPI time series that can be aggregated in different ways, and the chosen aggregation affects whether anomalies are visible. Expert labels for real incidents are scarce, so candidate aggregations cannot be ranked with standard supervised metrics. This thesis focuses on injecting synthetic anomalies into each candidate aggregation and using AUC-ROC on Matrix Profile discord scores as a label-free fitness signal.

Unexpected changes in ordinary traffic patterns or behaviors can occur at any instant, potentially resulting in network faults or quality of service problems. Telecom traffic is not a single univariate series; instead, it can be viewed under several categories. For example, a call can have different types, an event type may take many values, and a subscriber may belong to a given type. Traffic type, service type, partner-related information, and other categories create many candidate aggregations on which anomaly detection can be run.

Having a fully aggregated view can hide local anomalies, while monitoring every categorical combination separately exposes them only at high computational and interpretability cost. This creates a dilemma: which aggregation level should anomaly detection work on? Operators may use a fixed hierarchy or manual rules. For example, if user department or protocol type is available, anomalies can be detected at that level; aggregating by user protocol can make some anomalies visible. Running anomaly detection independently on every variable combination, however, is costly computationally and in terms of interpretability. Therefore, automatic search methods are needed to identify suitable candidate aggregations.

1.2 Objectives and Expected Results

This thesis aims to develop a new framework for anomaly detection in grouped telecommunications time series with hierarchical aggregation, using Grammatical Evolution to evolve programs from a formal grammar. From a data perspective, the aim is to work with the categorical dimensions of mobile network traffic records—call type, event type, subscriber type, traffic type, service type, partner-related information, and related fields—so that anomalies can be studied at global and local aggregation levels.

From a methodological point of view, the aim is to design a grammar-based evolutionary approach that automatically generates candidate hierarchical structures and

evaluates them by how well the resulting series support anomaly detection (e.g. ROC-AUC on Matrix Profile discord scores). Because anomaly labels are scarce, synthetic anomalies provide the evaluation signal. This assessment compares aggregation strategies and tests whether evolved or partial hierarchies improve ROC-AUC over fixed aggregation baselines.

1.3 Document Organization

The dissertation is organised into six chapters. Chapter 1 presents the motivation, objectives, and document organisation. Chapter 2 reviews the theoretical background: time series, hierarchical and grouped time series, anomaly detection, anomaly generation, Grammatical Evolution, and evaluation metrics. Chapter 3 examines the literature on anomaly detection in telecommunication networks, grouped and hierarchical time series, Matrix Profile-based anomaly detection, combinatorial optimisation for selecting time series or structures, and Grammatical Evolution as an optimisation method. Chapter 4 describes the telecommunications dataset, preprocessing and aggregation, synthetic anomaly injection, hierarchy construction, and the PonyGE2 implementation. Chapter 5 presents the experimental results, comparing aggregation strategies and examining the evolved hierarchical structures, strengths, and limitations. Chapter 6 summarises the contributions, limitations, and future work, including the use of real incident labels and online detection.

1.4 Methodological Approach

Aggregation choice is framed as a combinatorial search over categorical dimensions of telecom CDR/switch series; fixed views can hide local anomalies or make monitoring every combination costly. Candidate hierarchies are encoded in a BNF grammar so that only valid aggregation structures are generated. Grammatical Evolution (PonyGE2) then evolves aggregation programmes from that grammar. For each candidate, an aggregated series is built and scored with Matrix Profile (STUMPY) discord distances in a label-free way. Synthetic anomalies are injected, ROC-AUC on the discord scores is used as the fitness signal, and evolved or partial hierarchies are compared with fixed aggregation baselines. The implementation uses Mobai records, with preprocessing, aggregation, and anomaly injection, and lives in the BNF grammar plus PonyGE2.

1.4.1 Search Method

To support this master's thesis, a layered bibliographic search was used. The first step built a synonym/subtopic bank (grouped/hierarchical time series, telecom anomaly detection, Matrix Profile, combinatorial optimisation, Grammatical Evolution, anomaly injection) and ran Boolean queries in Google Scholar and IEEE Xplore, with additional checks on SpringerLink and other publisher sites as needed. The queries focused on keywords such as "Multivariate Time Series", "Grouped time series", "Hierarchical Time Series Forecasting", "Hierarchical anomaly detection", "Time Series and Anomaly Detection in Telecommunication", "Combinatorial Optimization", "Anomaly injection", and "Matrix Profile".

Because the integrated topic is sparse and hard to retrieve by keywords alone, Consensus and Perplexity were then used to surface additional candidate papers and related

terms. From a small set of highly relevant seed papers, “Cited by” and related-work links were followed to catch papers the synonym list missed.

Candidate articles were screened primarily by relevance to the thesis gap (telecom anomaly detection, aggregation/hierarchy, Matrix Profile/discord detection, combinatorial/evolutionary search, and Grammatical Evolution). Publication date (preferring work after 2015 where appropriate), venue quality, and citation impact were used as secondary signals, not as a fixed citation cut-off. Core databases for this stage were IEEE Xplore, Google Scholar, SpringerLink, MDPI, and PLOS; each retained paper was checked against the PDF before inclusion in the literature set.

Chapter 2

Theory and Background

2.1 Time Series

A time series is inherently systematic when observations follow a structured, consistent protocol for data collection and temporal indexing (Box et al., 2015). While traditional time series assume regular intervals (e.g., hourly, daily) to simplify analysis and modeling, irregularly spaced data (e.g., event-driven recordings) can also be understood as time series data in a broader sense. Variables within a time series may evolve over time (e.g., introducing new metrics in longitudinal studies), but methodological rigor requires consistency in measurement definitions to ensure comparability across observations (Lim and Zohren, 2021). Time series data is univariate if it contains a single variable, while in multivariate time series, additional variables may be incorporated, provided their temporal alignment and relevance are preserved.

2.1.1 Hierarchical Time Series

Time series is a sequence of data points indexed in time order, typically collected at regular intervals. Hierarchical time series are structured time series data sets arranged into hierarchical relationships. Typical hierarchies include geographical areas (e.g., national → state → city) or product categories (e.g., product group → SKU(Stock Keeping Unit)).

Hierarchical time series exhibit multi-level aggregation, where lower-level components combine through arithmetic operations (e.g., summation) to reconstruct higher-level series (R. J. Hyndman and G. Athanasopoulos, 2021). For example, consider a geographic hierarchy:

- **Level 1 (National)**: Total sales in a country.
- **Level 2 (Regional)**: Sales in the *North* and *South* regions.
- **Level 3 (Local)**: Sales in cities (e.g., *North: City A + City B*; *South: City C + City D*).

The national series equals the sum of regional series (National = North + South National=North+South), and each regional series equals the sum of its constituent cities. This coherence ensures consistency across levels (Wickramasuriya et al., 2019).

When analyzing hierarchical time series, it is possible to balance computational efficiency with granularity by allowing concurrent analysis at several aggregation levels (R. J. Hyndman and G. Athanasopoulos, 2018).

As shown in Figure 2.1, hierarchical time series arrange temporal data into several aggregation levels. The framework consists:

- **Root level**: The whole temporal sequence is represented by the aggregate series y_t for $t = 1, \ldots, T$
- **Intermediate levels**: Disaggregated elements (e.g., $y_{A,t}$ and $y_{B,t}$) generating partitions of the root series
- **Leaf level**: The most detailed constituents (e.g., $y_{AA,t}$, $y_{AB,t}$, $y_{AC,t}$ under node A; $y_{BA,t}$, $y_{BB,t}$ under node B)

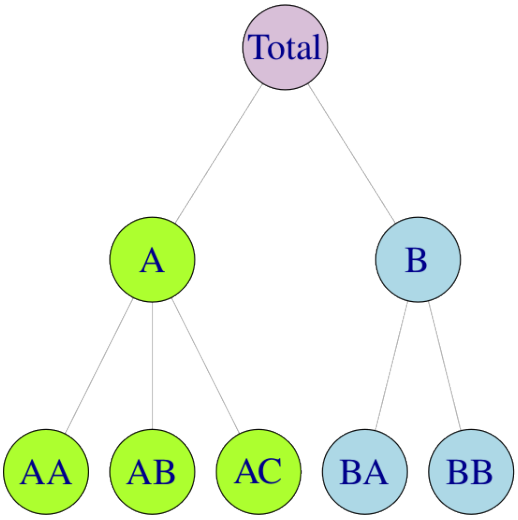


Figure 2.1: A two level hierarchical tree diagram (R. J. Hyndman and G. Athanasopoulos, 2018).

Approaches to Hierarchical Forecasting

Forecasting within a hierarchical system uses techniques to ensure that forecasts are credible and internally coherent. This implies that the aggregation of forecasts at one level should match the forecast of the level above. The main approaches used here are bottom-up, top-down, and middle-out.

The bottom-up approach begins its predictions at the base level and aggregates them to higher levels. This approach is computationally reasonably practical and performs well with steady, consistent upper-level datasets. The top-down approach starts at the summit of the hierarchy. Analysts distribute forecasted numbers across the hierarchy using historical ratios, apportioning whole numbers to each section. The middle-out approach connects bottom-up and top-down methods. It aims to neutralize the shortcomings of both approaches while maximizing their benefits. By focusing on a mid-level, one can ensure the accuracy of estimates at both top and bottom levels using a much more exact balance (R. J. Hyndman and G. Athanasopoulos, 2018).

2.1.2 Grouped Time Series

Grouped time series represent collections of temporally structured data organized by shared attributes (e.g., device type, customer demographics) rather than hierarchical

relationships. Unlike hierarchical series, grouped data lacks inherent aggregation constraints. Forecasts for individual series are developed independently and then analyzed collectively for cross-sectional patterns (R. J. Hyndman and G. Athanasopoulos, 2018). Grouped time series significantly help macroeconomic analysis (George Athanasopoulos et al., 2017) and supply chain forecasting (Wickramasuriya et al., 2015), where multiple cross-sectional dimensions demand coherent alignment, free from strict hierarchical dependence.

Often, disaggregating factors are both nested and crossed. For example, Australian tourism data can be disaggregated by hierarchical geographical categories (states, and regions nested within states), but also by the four purposes of travel: holiday, business, visiting friends and relatives, and other. This grouping variable does not nest within any of the geographical variables.

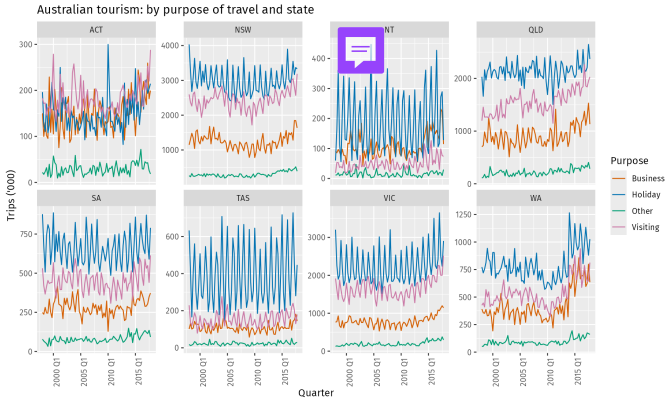


Figure 2.2: Australian domestic overnight trips over the period 1998 Q1 to 2017 Q4 disagrgreagted by purpose of travel and by state (R. J. Hyndman and G. Athanasopoulos, 2021).

Grouped time series significantly help macroeconomic analysis (George Athanasopoulos et al., 2017) and supply chain forecasting (Wickramasuriya et al., 2015), where multiple cross-sectional dimensions demand coherent alignment, free from strict hierarchical dependence.

2.2 Anomaly Detection

In machine learning, anomaly detection—which aims to find patterns that deviate significantly from expected behavior—is a fundamental task. This process is essential in many fields, including fault diagnosis (Chandola et al., 2009), network security, and fraud detection. The main objective of anomaly detection is to differentiate between normal and aberrant events within a dataset Univariate anomaly detection in mathematical deviation analysis identifies outliers in datasets with only one variable.

2.3 Anomaly Generation

Anomaly generation is the methodical production of irregular patterns or outliers that significantly depart from normal data distributions (Hawkins, 1980). This process has two

uses in machine learning settings: stress-testing system robustness (Chandola et al., 2009) and enhancing detection algorithms through augmented training data. Modern methods stress the need of generating semantically plausible contextual anomalies in particular application domains (Ruff et al., 2021).

2.3.1 Rule-Based Approaches

Expert-defined constraints in domain-specific anomaly generation break normal system behavior (Chandola et al., 2009).

2.3.2 Validation of Synthetic Anomalies

To ensure synthetic data accurately reflects real-world distributions and contextual relevance, validation requires a multi-faceted approach, human expert evaluation verifies semantic validity and practical utility (Ruff et al., 2021), and ROC-AUC (Area Under the Receiver Operating Characteristic Curve) benchmarks detection performance by measuring a model's ability to distinguish anomalies or patterns across classification thresholds (Hand and Till, 2001) in a dataset of targets with categorical variables.

2.4 Grammatical Evolution

2.4.1 Combinatorial Optimization

Combinatorial optimization, a fundamental paradigm in discrete mathematics, focuses on identifying optimal configurations within finite solution spaces and provides formal frameworks for NP-hard problems (C. H. Papadimitriou and K. Steiglitz, 1982). Its historical roots trace back to Euler's 18th-century graph theory (Euler, 1741) and Monge's pioneering work on optimal transportation (Monge, 1781). The 20th century witnessed pivotal advancements, including the Hungarian algorithm for assignment tasks (Kuhn, 1955), Dantzig's linear programming breakthroughs (Dantzig, 1990), and Cook's formalization of computational complexity theory (Cook, 1983). Landmark contributions by (Garey and Johnson, 1979) established the NP-completeness framework, classifying problems like the traveling salesperson problem (TSP) as computationally intractable, which spurred the development of exact methods such as dynamic programming and branch-and-bound algorithms (Applegate et al., 2007).

Genetic algorithms, simulated annealing, and ant colony optimization are heuristic and metaheuristic solutions to NP-hard problems (Glover and Greenberg, 1989; Zlochin et al., 2004). Frequently generating almost perfect solutions within reasonable computational times, these methods effectively explore solution spaces with stochastic processes.

Combinatorial Optimization Algorithms

One can generally classify combinatorial optimization techniques into exact algorithms and approximations. Although precise solutions are guaranteed, they could be computationally costly for important problem contexts. These include integer programming, branch-and-bound dynamic programming, and (Christos H Papadimitriou and Kenneth Steiglitz, 1998). On the other hand, approximation methods provide almost perfect solutions within polyn time. Hence, they fit large-scale problems where exact solutions are

impractical (Vazirani, 2001). Among the examples are greedy algorithms, local search approaches, and metaheuristic techniques, including simulated annealing and genetic algorithms.

2.4.2 Genetic Programming

Genetic programming (GP) is an evolutionary algorithm that first operates iteratively on a population of candidate solutions, imitating natural selection processes to evolve solutions to problems. An operator of selection utilizes fitness measures to choose individuals for reproduction, while crossover mix elements of differents solutions selected for reproduction, to give origin to new offspring. The offspring are then mutated to introduce random variations, and the process continues until a stopping criterion is met, such as reaching a maximum number of generations or achieving a satisfactory solution. Finally, the operator of survival selects the best individuals to form the next generation, ensuring that the population evolves over time.

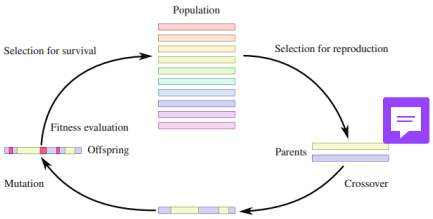


Figure 2.3: Generational loop in an evolutionary algorithm. From a population of solutions, symbolized here by colored sticks, one selects individuals who reproduce by crossover and mutation. The offspring thus generated is evaluated and incorporated into the population. Ultimately, individuals are eliminated by a selection operator for survival to bring the population back to its initial si

Genetic programming (GP) is a specialized form of genetic algorithms that evolves computer programs to solve problems (Koza, 1992). GP solutions, represented as tree structures, e.g., each node corresponds to a function or terminal. The evolution process generate new program variants. GP is particularly effective for symbolic regression, automated algorithms discovery, and optimization tasks (Koza, 1992). It can discover complex relationships in data, automate the design of algorithms, and optimize solutions across various domains, including finance, engineering, and artificial intelligence (Banzhaf et al., 1998). The population and numbers of generations are crucial parameters in GP, influencing the search space exploration and convergence speed. A larger population size allows for more diverse solutions, while an appropriate number of generations balances exploration and exploitation.

Grammatical Evolution

The method was first introduced by Dr. Michael O'Neill in 2001 as a way to apply evolutionary computing methods to problems with a Formal Grammatical Structure. A Formal Grammar can be defined as a set of rules that define a language's syntax, allowing for the generation of valid strings or structures within that language. In English, we start learning the vocabulary and grammar rules of the language, which allows us to form sentences and communicate effectively. Similarly, in programming, a formal grammar defines

the syntax and structure of a programming language, enabling the creation of valid code structures. In Grammatical Evolution (GE), the genotype ~~is~~ is an integer vector, instead of a tree structure, which is used to represent the grammar rules. The Grammar maps those vectors to ~~a~~ valid phenotypes. The representation with integers allows for a more flexible and efficient approach to genetic programming, as it can easily adapt to different grammar structures and languages.

The process of Grammatical Evolution involves the following steps:

1. Define a BNF representation of the grammar;
2. Randomly generate genotypes as integer vectors;
3. Evaluate the fitness of genotype by mapping the genotypes to the phenotypes and testing against the fitness function;
4. Use Genetic Operators such as Crossover and Mutation to evolve the population of gynotypes;
5. Repeat steps 3 and 4 until a satisfactory solution is found or a stopping criterion is met.

2.4.3 Backus-Naur Form

A BNF grammar is a formal way to describe the syntax of a language, allowing for the generation of valid strings or structures within that language. It is represented by a tuple containing 4 sets:

- T: The terminal set
- N: The non-terminal set
- P: Set of production rules
- S: Start symbol (which is a member of N)
- ~~T: The terminal set~~
- ~~N: The non-terminal set~~
- ~~P: Set of production rules~~
- ~~S: Start symbol (which is a member of N)~~

For example, consider a simple grammar that describes a mathematical expression with addition, subtraction, multiplication, and division, using the variables x and y.

$$T = \{+, -, /, *, x, y\}$$

So the terminal set is the set of all operators and variables you want to use in your program.

$$N = \{\langle e \rangle, \langle o \rangle, \langle v \rangle\}$$

The non-terminal set is the set of all production activities that the program can undertake. In this situation, $\langle e \rangle$ is an expression, $\langle o \rangle$ is an operator, and $\langle v \rangle$ is a variable.

P then is the set of production rules:

$$\langle e \rangle ::= \langle e \rangle \langle o \rangle \langle e \rangle \mid \langle v \rangle \tag{2.1}$$
$$\langle o \rangle ::= + \mid - \mid / \mid * \tag{2.2}$$
$$\langle v \rangle ::= x \mid y \tag{2.3}$$

Notice that all entries in P are entries of either T or N. The pipe operator $\mid$ represents *or*. So when an $\langle e \rangle$ is encountered in the genotype, it is either replaced with $\langle e \rangle \langle o \rangle \langle e \rangle$, or $\langle v \rangle$, and likewise for $\langle o \rangle$ and $\langle v \rangle$.

Finally, the start symbol:

$$S = \langle e \rangle$$

The simplest way of describing genotypes is as a non-negative integer array. For example, considering the following genotype:

$$C : [4, 15, 75, 8, 41, 12]$$

The starting sequence is simply the start symbol:

$$\langle e \rangle$$

The general rule for evaluating a genotype is as follows:

$$\text{selected_option} = \text{gene_value} \bmod N$$

where:

- gene_value is the current integer from the genotype,
- N is the number of production choices available for the non-terminal,
- mod denotes the modulo operation,
- selected_option is the index of the chosen production rule.

The first value of the genotype is 4, and there are 2 options for $\langle e \rangle$ in the production rules $4 \bmod 2 = 0$, so we take the 0-th (zero-indexed, the 1st entry) option from the production rules, $\langle e \rangle \langle o \rangle \langle e \rangle$.

Then the expression has expanded to:

$$\langle e \rangle \langle o \rangle \langle e \rangle$$

Then, the first entry evaluated $\langle e \rangle$ (it is not a member of the terminal).

There are 2 options for $\langle e \rangle$ in the production rules. The 2nd entry in the genotype, 15, and $15 \bmod 2 = 1$, so the 1st (zero-indexed, 2nd entry) option from the production rules, $\langle v \rangle$.

Continues the following way:

1. **Full Expression:** $\langle v \rangle \langle o \rangle \langle e \rangle$

- Current Expression: $\langle v \rangle$
- Number of $\langle v \rangle$ options in Grammar: 2
- genotype value: 75
- $75 \bmod 2 = 1$
- Replace $\langle v \rangle$ with y

2. **Full Expression:** $y\langle o \rangle\langle e \rangle$
   - Current Expression: $\langle o \rangle$
   - Number of $\langle o \rangle$ options in Grammar: 4
   - genotype value: 8
   - $8 \bmod 4 = 0$
   - Replace $\langle o \rangle$ with $+$
3. **Full Expression:** $y + \langle e \rangle$
   - Current Expression: $\langle e \rangle$
   - Number of $\langle e \rangle$ options in Grammar: 2
   - genotype value: 41
   - $41 \bmod 2 = 1$
   - Replace $\langle e \rangle$ with $\langle v \rangle$
4. **Full Expression:** $y + \langle v \rangle$
   - Current Expression: $\langle v \rangle$
   - Number of $\langle v \rangle$ options in Grammar: 2
   - genotype value: 12
   - $12 \bmod 2 = 0$
   - Replace $\langle v \rangle$ with x
5. **Final Expression:** $y + x$

2.4.4 Initialization Phase

The initialization phase can be seen as the first step in the evolutionary process, where is done in genome or in derivation tree. To unable to control the deep, focus in one derivation tree, Position Independent Grow method, which try generate individuals to fullfill the maximum depth of the tree: not all too deep, not all too shallow. PIG utilizes random order expansion instead of the traditional left-first derivation approach. This method ensures that at least one branch reaches the maximum depth. This approach offers two key advantages: it reduces the occurrence of duplicate structures and generates more diverse trees in the initial population.

2.4.5 Selection Mechanism

The objective of the selection mechanism is to choose individuals from the population to reproduce, which means to be parents for the next generation, also to preserve diversity, is plausible to have some ramdomness in the selection process. The selection mechanism is based on tournament selection, which is a widely used method in genetic algorithms. It envolves ramdomly select size of tournament individuals from the population, then identify and retain the best individual from each tournament. This process is repeated until size of generation parents are selected. The system incroporates elitism to ensure that the best individuals from the previous generation are preserved in the next generation, enhancing the overall quality of the population.

2.4.6 Variation Operators

The variation operators are designed to explore the search space by modifying genetic material. The system employs two primary operators:

Crossover and mutation. These operators are crucial for generating new individuals in the population, allowing the algorithm to explore diverse solutions. For the crossover operator, variable one-point crossover is used from many others, which involves swapping tails after randomly selected cut points. This operator allows the genome length to vary after crossover, enabling the combination of beneficial traits from parent individuals. Mutation is applied to every child after crossover, using a codon-based integer flip method ~~for better operational effectiveness~~. This operator randomly flips each codon with specified probabilities, ensuring that only the used portions of the genome are mutated. The mutation probability is defined as:

$$P_{\text{mutation}} = \frac{1}{\text{genome_length}} \tag{2.4}$$

This approach ensures that the mutation process is efficient and does not introduce excessive noise into the population, allowing for meaningful exploration of the search space.

2.4.7 Replacement Strategy

The replacement strategy is a critical component of the evolutionary process, determining how new individuals are integrated into the population. In this system, the replacement strategy is designed to maintain diversity while ensuring that the best solutions are preserved.

The replacement strategy is based on generational replacement with elitism, which ensures that the best individuals from the previous generation are retained in the next generation. This approach allows the algorithm to converge towards optimal solutions while maintaining genetic diversity within the population. The remaining individuals in the population are replaced with newly generated individuals, ensuring that the population evolves over time. This strategy allows the algorithm to explore new areas of the search space while retaining the best solutions found so far.

2.5 Metrics

2.5.1 Precision and Recall

In anomaly detection, precision and recall provide complementary perspectives on model performance, particularly necessary when class imbalances skew evaluation (e.g., rare fraud cases amidst billions of legitimate network events). These metrics are defined as follows:

Precision

Precision quantifies a model's ability to minimize false positive classifications:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.5}$$

Where:

- TP = True Positives
- FP = False Positives

Recall evaluates comprehensive positive case identification:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.6}$$

Where:

- FN = False Negatives

2.5.2 Trade-off and Applications

The inherent trade-off between precision and recall manifests through classification threshold adjustments—increasing recall typically decreases precision and vice versa. This relationship necessitates domain-specific optimization.

High recall becomes paramount in medical diagnostics, for example, where failing to detect actual positive cases (e.g., cancerous tumors) can have severe consequences. Meanwhile, in spam detection, high precision is crucial to avoid misclassifying legitimate emails as spam, which can lead to significant information loss and user frustration.

F1 Score

The F1 score harmonizes precision and recall through their harmonic mean:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.7}$$

The equation provides a singular performance measure for balanced class distributions. However, interpretation requires caution in severely imbalanced c prove more appropriate.

2.5.3 Area Under the ROC Curve (AUC)

The ROC curve plots the true positive rate against the false positive rate across different classification thresholds (Hanley and McNeil, 1982). One performance metric for assessing binary classification systems, especially when there is class imbalance, is the Area Under the ROC Curve (AUC-ROC) (Fawcett, 2006). AUC measures the classifier's capacity to discriminate between classes.

Interpretation and Calculation

AUC values range from 0 to 1, where:

- 0.5 indicates random guessing
- 1 represents perfect separability
- Values below 0.5 suggest inverse prediction tendencies

The Wilcoxon-Mann-Whitney statistic provides a non-parametric AUC estimation method (Hanley and McNeil, 1982):

$$\text{AUC} = \frac{\sum_{i=1}^{n_+} \sum_{j=1}^{n_-} \mathbf{1}(s_i^+ > s_j^-)}{n_+ \cdot n_-} \tag{2.8}$$

where n_+/n_- denote positive/negative instances and s represents classifier scores.

Advantages and Limitations

AUC provides robustness against class imbalance and threshold-agnostic evaluation (Bradley, 1997). However, it fails to capture cost-sensitive decision contexts and may be misleading when comparing classifiers with different ROC curve crossings (Lobo et al., 2008).

Chapter 3

Literature Review

3.1 Anomaly Detection in Telecommunications Networks

Telecommunication anomaly detection is difficult because labels are scarce. Moreover, the industry produces large amounts of data, with few viable labels that can be identified as anomalies. Often, the problem is not detecting the anomaly but selecting which subsection of the network window to monitor.

Gonzalez et al. (2024) show that the modern multivariate detector is actually modest at detecting in a ~~fixed~~, fully aggregated view where the labels are sparse. Of 730,000 samples, only 9,000 were labeled using a fixed set of 12 features. Using the DC-VAE, the mean range-F1 is around 52%.

Burgueno et al. (2020) indicate that the network had 25,000 mobile base station coverage areas. Moreover, for each cell, they can measure 240 features, but the study focuses on only 15 of the most important. From this perspective, manual selection is a bottleneck because the 225 unmonitored features may still contain relevant information; for example, anomalies may occur in an unmonitored signal.

CDR and mobile activity records are recognized for analysis in telecom monitoring. Sultan et al. (2018) focused on traffic prediction rather than anomaly detection, and no detection metrics are reported.

In the Orange Paris case, features are manually selected to construct hand-designed aggregations (Lemaire et al., 2024). Moreover, the best precision they achieve is 28.63% for matching real events, indicating that many alarms were false positives. The system is actually useful but noisy.

Existing work assumes a fixed aggregation and therefore may miss anomalies that would only appear under a different grouping, so manual feature selection does not scale. Automated search is useful when the best aggregation is not obvious in advance; in those papers, there is a fixed review of aggregation, but they do not address how to automate the aggregation of time series. The next section will be about this topic.

3.2 Anomaly Detection in Grouped Time Series

Anomaly detection in grouped time series treats the detector input as a chosen representation, not the whole network. The dataset is aggregated by related features/entities at the aggregation level, rather than treating a network as a flat collection of features. It is

important because the detection quality is dependent on the aggregation of features, not only the detector.

Mefleh et al. (2025) used an unsupervised Mahalanobis distance approach in a telecommunications setting. They got an improvement from 0.77 on the raw features to 0.86 when the denominator information is included, then to 0.93, adjusting the ratios, and finally to 0.99 with learned adjustments. This result shows that detection quality changes with feature transformation and aggregation.

Hua et al. (2022) show that the base stations share some operational behavior, which the model can learn and train to detect other anomalies in the other stations. GenAD was pre-trained on 3,000 unlabeled base stations, using multivariate time series features from China Mobile base stations. Their F1 score is 0.953 (precision 0.910). However, their evaluation used point-adjusted segment scoring, treating anomaly detection as an entire time interval, not as a separate timestamp. So the performance across grouped time series studies is not directly comparable because data sets, anomaly labels, and evaluation metrics differ.

In the study by Oguz and Kalaycioglu (2022), they simulate a multi-tier network that includes aggregations from the user equipment measurements and classifies the state of each femtocell access point. Performance changes with the region of interest radius, signal shadowing, and aggregation methods. At a 30 m region of interest with 2 dB shadowing, the 1D CNN achieved 80.1% accuracy, but as ROI size and the shadowing increase, the accuracy decreases, so this shows that including more observations is not necessarily beneficial when added signals are less relevant or noisier.

Based on observations, we can extract that grouping ~~and representations~~ can strongly affect anomaly detection performance. Although aggregations are selected manually before modeling the relevant KPIs, entities, aggregation levels, and spatial boundaries, none of them systematically search for an aggregation structure that best characterizes the anomaly. ~~So we will explore how grammatical evolution will search from a grammar-defined structure space of valid hierarchical aggregation of telecommunication data, rather than selecting the fixed group time series.~~

3.3 Anomaly Detection in Hierarchical Time Series

The main difference between grouped time series and hierarchical time series is that grouped time series are a chosen representation. Whereas in the hierarchy, the anomaly sits inside the structure. ~~In telecom, an anomaly alert is not enough because the operator also needs to know where the anomaly appears in the network or service structure. Hierarchy provides that location information; grouping does not.~~

Hierarchy in anomaly detection is important because an alert such as “traffic is abnormal” is not enough for a telecom operator to act on. The operator must determine whether it affects the entire network, a region or specific cell of the switch, or only one KPI category. So this top-down approach helps break down into smaller branches; although that hierarchy is not necessarily the best aggregation for detecting anomalies.

Hong et al. (2012) detect and localize anomalies in hierarchical ISP call-detail data. It analyzes customer-care call logs and signaling call records across predefined hierarchies, allowing operators to identify not only that an abnormal behavior occurs but also the level at which it appears. They found over 94% of reference anomalies, and separately reported 99.7% anomaly identification accuracy. These results were obtained from a fixed additive hierarchy and Holt–Winters spike forecasting. So the author demonstrates hierarchical

localization rather than searching over alternative aggregation structures. It does not search for alternative partial aggregations or score non-spike discords, which is the gap addressed by this thesis.

On the other hand, Wolff et al. (2022) treat anomaly localization as the sequential search problem of a known tree of processes. Instead of measuring every leaf process equally, their hierarchical dynamic search method first samples aggregate internal nodes and then moves towards the branch most likely to contain an anomaly. This ~~can be useful for the thesis because it~~ shows that hierarchy can be an active search space, not merely a reporting structure used after detection. However, Wolff assumed tree topology is already known, as search is only within it. This differs from our approach, which uses a GE to generate candidate aggregations before applying the anomaly scorer.

Huang et al. (2026) propose HYDRA, a multi-level hierarchy-driven anomaly detector that ranked first among 40 algorithms across 40 TSB-AD datasets.

3.4 Matrix Profile and Time-Series Discord Detection

Matrix Profile is introduced here because the GE generates many aggregations, and for each produced candidate series, GE needs a scorer. Also, MP is exact, label-light, and computationally feasible for many candidates. It takes one candidate time series and computes the nearest-neighbour distances for each subsequence and returns a discord score.

A time-series discord is the subsequence most distant from its nearest non-self match (Keogh et al., 2005). On the Matrix Profile, the discord is the position with the maximum nearest-neighbour distance (Yeh et al., 2017). For a short and unusual subsequence or spike, the discord may capture well, but may poorly capture a level shift, which is a sustained change to a new level; a ramp, which gradually drifts upwards over time; or a multivariate contextual anomaly, where each series looks normal alone but the relationship is abnormal. Computational efficiency is substantially improved: a refinery time-series self-join is reduced from an estimate of 153.8 days by brute force to 1.2 hours with STAMP/STOMP (Yeh et al., 2017). Also, Zhu et al. (2016) showed GPU-STOMP could process 100 million subsequences in about 12 days, compared to the estimate of 1585 years for brute force. STUMPY (Law, 2019) gives the practical implementation of this process in Python. ~~Although Matrix Profile can score a candidate, it does not know which candidate aggregation to score. That is why we are moving on to combinatorial optimisation.~~

3.5 Combinatorial Optimisation for Selecting Time Series

Feature/subset selection is complex, so combinatorial optimisation will search a space of many candidates. The chosen set is large enough that the enumeration fails because the space is 2^n (Xue et al., 2016); you also need to search for the candidates and score each one by the fitness function.

Linares-Barrera et al. (2024) select lagged forecasting features on an electricity dataset, reducing the feature set from 7,704 to 3,813, with MAE improved from 207.193 to 194.400,

supporting representation pruning.

Contreras et al. (2024) study financial direction prediction with modest AUROC ($\sim$0.58–0.62), using GA with AUROC fitness, where each candidate feature subset is scored.



Chapter 4

Materials and Methodology

4.1 Problem Formulation

This research develops a hierarchical anomaly detection framework for mobile network data to improve service reliability. The study focuses on grouped time series from a network switch (July–Oct 2022), analyzing 4.2M hourly records of call entries/exits. Two core metrics are tracked: #Events (call frequency) and Event Duration (total call time), aggregated across six dimensions like traffic type (ON/OFF-NET) and subscriber category (Roaming/Postpaid).

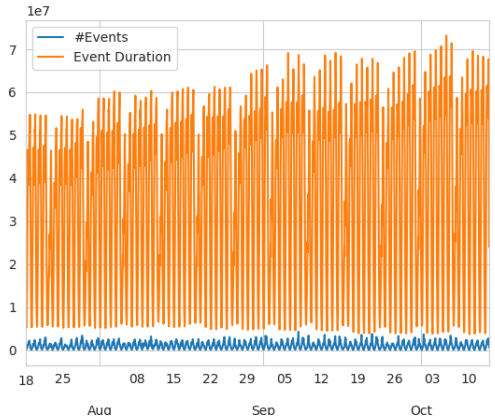


Figure 4.1: Hourly network traffic patterns showing (A) Total #Events (left axis) and Event Duration in hours (right axis).

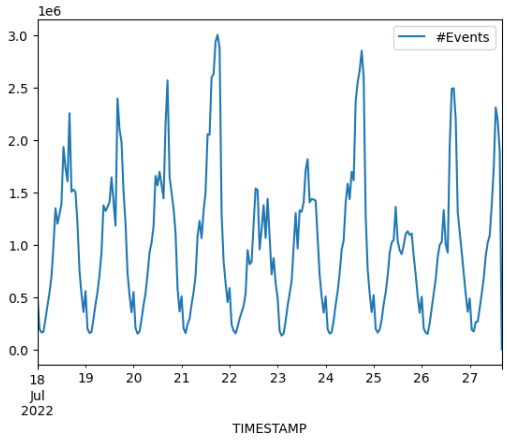


Figure 4.2: Hourly call event frequency.

Table 4.1: Cardinality and Dominant Categories in Vodafone Data

Variable	Unique Values	Top Category	Frequency
Call Type	1,555	VOICE	849 (54.6%)
Event Type	1,555	ORIGINATED	716 (46.0%)
Subscriber Type	1,555	Roamer In	1,486 (95.6%)
Traffic Type	1,555	INT	1,486 (95.6%)
Service Type	1,555	NOT-ROAMING	1,544 (99.3%)
Partner Description	1,555	P_0001	65 (4.2%)

4.1.1 Key Observations

The categorical variable analysis reveals three distinct distribution patterns:

- **Highly Skewed Variables**:
  - Subscriber Type: 95.6% Roamer In
  - Traffic Type: 95.6% INT
  - Service Type: 99.3% NOT-ROAMING
- **Balanced Variables**:
  - Call Type: 54.6% VOICE vs 45.4% SMS
  - Event Type: 46.0% ORIGINATED vs 54.0% other
- **High-Cardinality Variable**:
  - Partner Description: 1,555 unique values (long-tail distribution)
  - Top partner P_0001 covers only 4.2% of data

4.1.2 Analytical Implications

$$\text{Information Gain} = -\sum_{i=1}^{n} p_i \log_2 p_i \tag{4.1}$$

Where p_i is the proportion of each category. Calculated entropy values:

Table 4.2: Information Content by Variable

Variable	Entropy (bits)
Call Type	0.996
Event Type	1.000
Subscriber Type	0.286
Traffic Type	0.286
Service Type	0.048
Partner Description	7.602

The high entropy of Partner Description (7.6 bits) suggests it requires special handling during hierarchical aggregation to avoid overfitting. Conversely, near-zero entropy of Service Type indicates limited predictive value for anomaly detection.

4.2 Adopted Methodology

The original dataset contains approximately 1M records with six features, filtering and removing empty timestamps. The aggregation process involves filtering out empty timestamps, aggregated by all combinations of the six features with the distinct values of each features. The aggregation is performed using the #Events as the target variables, resulting in a new dataset with 1,555 unique combinations of features out of 94080 availables. The mean of timestamps instances in all possibilities of not empty combinations is 569, witch means that the dataset is highly sparse. sera isto? To create anomalies, 5% of the timestamps are randomly selected and modified to include synthetic anomalies. Finally, the dataset is then split into training (80%) and testing (20%) sets.

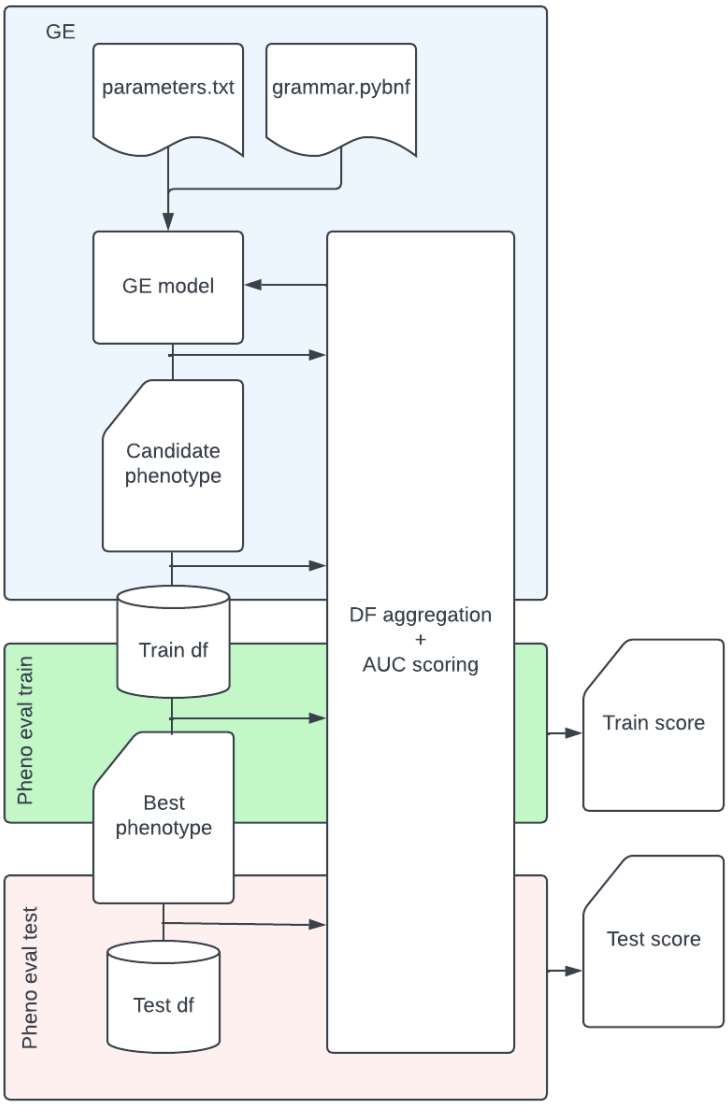


Figure 4.3: Hierarchical anomaly detection pipeline combining data aggregation, synthetic anomaly injection, and genetic model optimization.

Workflow Stages:

1. **Data Preparation**
   - Input: 1M records (6 features)
   - Filter: Remove empty timestamps
   - Aggregate: Random feature combinations
2. **Anomaly Injection**
   - Insert 5% synthetic timestamp anomalies
   - Quality Check: "Verify anomaly detection effectiveness"
3. **Data Partitioning**

- Split into training (80%) and testing (20%) sets

4. **Model Development**
    - Grammar-based evolution
    - Inputs: Rule templates + Parameters
5. **Evaluation & Refinement**
    - Output: Best-performing detection rules
    - Metrics: AUC-ROC scores
    - Iterative parameter tuning

Key Processes:

- Hierarchy parsing → Condition filtering → Temporal aggregation
- Matrix Profile analysis → Weighted AUC calculation
- Data gap filling → Event count normalization

Symbol Legend:

- $\rightarrow$: Process flow direction
- **AUC** : Area Under Curve
- **DF** : dataframe

4.3 Model Development and Evaluation

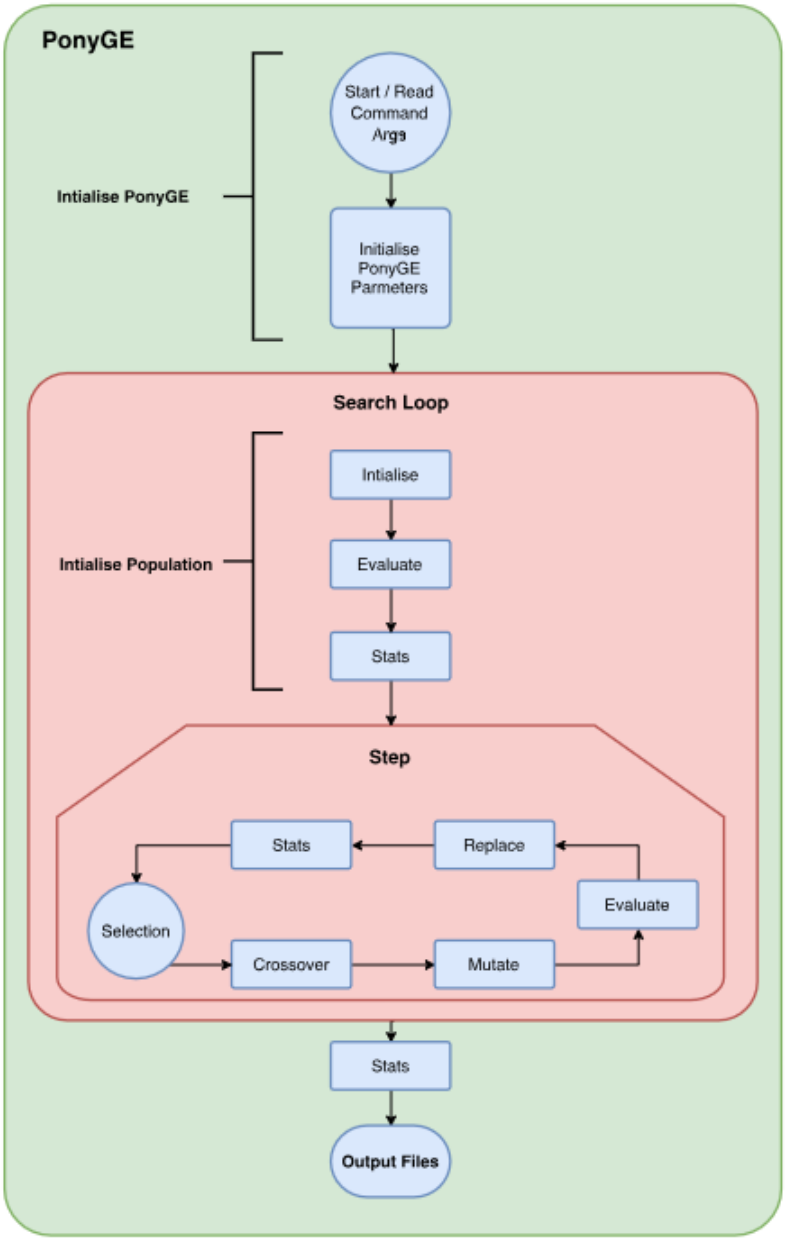


Figure 4.4: PonyGE2 control flow diagram (Fenton et al., 2017).

4.3.1 Genome Structure

The genome structure is defined by codons, for example each codon is an integer in the range $[0, 99{,}999]$, corresponding to a codon size of 100,000. Each codon value determines the selection of a grammar rule during program construction. Define max genome lenght is crucial for preventing the infinite growth of solutions, which represents the "DNA length".

4.4 Anomaly Injection

The anomaly injection process generates synthetic anomalies in the dataset to simulate real-world scenarios. Since the dataset does not contain anomalies, this process is essential for evaluateing the effectiveness of the anomaly detection algorithms. The procedure involves defining anomaly parameters, selecting feature combinations, and modiflying records to create anomalies. The goals of this process are to ensure that the anomalies are realistic and to maintain the integrity of the dataset while introducing anomalies.

For anomalie generation, we define the window size as 168 hours (1 week) and use a threshold of $\mu \pm 3\sigma$ based on the historical baseline. Each anomaly is generated by randomly selecting new features based on probabilities of less than 50% being choosed for each feature and the value of it randomly selected within the feature selected, to form a combination of categorical features (e.g., Service=ROAMING, Traffic Type=INT), then modifying the corresponding records within a 168-hour window. For each selected group, the anomaly value for #Events is set outside the normal range ($\mu \pm 3\sigma$), and an anomaly flag is assigned. This ensures anomalies are contextually realistic and distributed across diverse subsets.

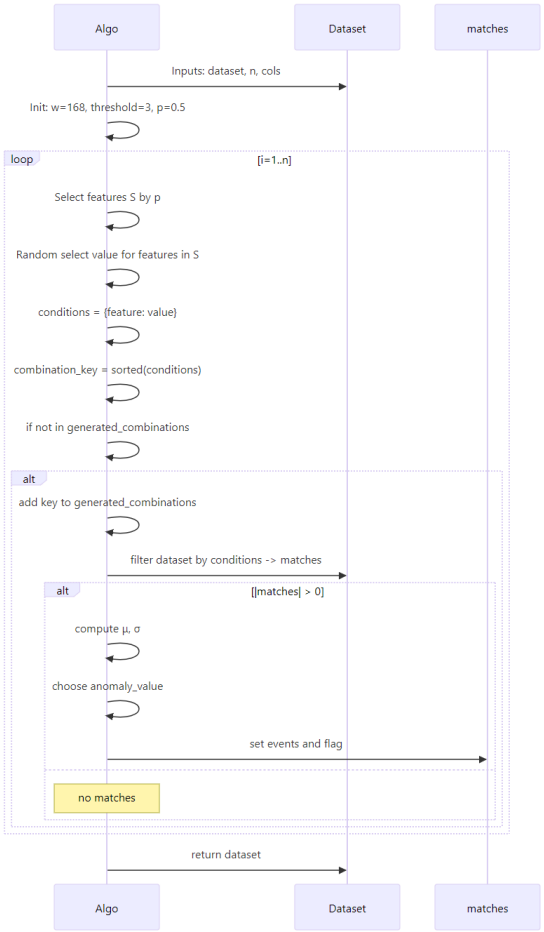


Figure 4.5: Anomaly injection cicle.

4.5 Hierarchical Data Structures

There are not a unique way to aggregate the data, and the aggregations can be limited by the filters applied. The filters can be empty or not, using the list of not empty filters of combinations of features with the distinct values of each feature, this allows us to limited the aggregations that can be made, proventing the empty timestamps.

The data is represented as a list of dictionaries, where each dictionary contains the values of the features and the target variable (#Events).

4.6 Genetic Programming Implementation

The genetic programming implementation is based on the PonyGE2 framework, which has the following key components:

Table 4.3: Key Configuration Parameters

Parameter	Value
CODON_SIZE	100000
CROSSOVER	variable_onepoint
CROSSOVER_PROBABILITY	0.75
GENERATIONS	20-100
MAX_GENOME_LENGTH	500
INITIALISATION	PI_grow
MAX_INIT_TREE_DEPTH	5
MAX_TREE_DEPTH	10
MUTATION	int_flip_per_codon
POPULATION_SIZE	10-50
FITNESS_FUNCTION	hierarchy_auc
REPLACEMENT	generational
SELECTION	tournament
TOURNAMENT_SIZE	2

Chapter 5

Results and Discussion

5.1 Performance Analysis

5.2 Time Series Visualization

5.3 Genetic Programming Analysis

Chapter 6

Conclusions and Future Works

6.1 Conclusions

6.2 Future Work

Bibliography

Applegate, D. L. et al. (2007). *The traveling salesman problem: A computational study*. Princeton University Press.

Athanasopoulos, George et al. (2017). "Forecasting with temporal hierarchies". In: *European Journal of Operational Research* 262.1, pp. 60–74.

Banzhaf, W. et al. (1998). *Genetic Programming: An Introduction*. Morgan Kaufmann.

Box, George EP et al. (2015). *Time series analysis: Forecasting and control*. 5th. John Wiley & Sons.

Bradley, Andrew P (1997). "The use of the area under the ROC curve in the evaluation of machine learning algorithms". In: *Pattern recognition* 30.7, pp. 1145–1159.

Burgueno, Jesus et al. (2020). "Online Anomaly Detection System for Mobile Networks". In: *Sensors* 20.24, p. 7232. DOI: 10.3390/s20247232.

Chandola, Varun, Arindam Banerjee, and Vipin Kumar (2009). "Anomaly detection: A survey". In: *ACM computing surveys (CSUR)* 41.3, pp. 1–58.

Contreras, Rodrigo Caetano et al. (2024). "Genetic Algorithm for Feature Selection Applied to Financial Time Series Monotonicity Prediction: Experimental Cases in Cryptocurrencies and Brazilian Assets". In: *Entropy* 26.3, p. 177. DOI: 10.3390/e26030177.

Cook, Stephen A. (1983). "An overview of computational complexity". In: *Communications of the ACM* 26.6, pp. 400–408.

Dantzig, George B. (1990). "Origins of the simplex method". In: *ACM SIGPLAN Notices* 28.3, pp. 141–151.

Euler, Leonhard (1741). "Solutio problematis ad geometriam situs pertinentis". In: *Commentarii academiae scientiarum Petropolitanae* 8, pp. 128–140.

Fawcett, Tom (2006). "An introduction to ROC analysis". In: *Pattern recognition letters* 27.8, pp. 861–874.

Fenton, Michael et al. (2017). "Ponyge2: Grammatical evolution in python". In: *Proceedings of the genetic and evolutionary computation conference companion*, pp. 1194–1201.

Garey, M. R. and D. S. Johnson (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W.H. Freeman and Company.

Glover, Fred and Harvey J Greenberg (1989). "New approaches for heuristic search: A bilateral linkage with artificial intelligence". In: *European Journal of Operational Research* 39.2, pp. 119–130.

Gonzalez, Guillermo Garcia et al. (2024). "One Model to Find Them All: Deep Learning for Multivariate Time-Series Anomaly Detection in Mobile Network Data". In: *IEEE Transactions on Network and Service Management* 21.2, pp. 1601–1616. DOI: 10.1109/TNSM.2023.3340146.

Hand, David J and Robert J Till (2001). "A simple generalisation of the area under the ROC curve for multiple class classification problems". In: *Machine learning* 45, pp. 171–186.

Hanley, James A and Barbara J McNeil (1982). "The meaning and use of the area under a receiver operating characteristic (ROC) curve." In: *Radiology* 143.1, pp. 29–36.

Hawkins, Douglas M (1980). *Identification of outliers*. Vol. 11. Springer.

Hong, Chi-Yao et al. (2012). "Tiresias: Online Anomaly Detection for Hierarchical Operational Network Data". In: *2012 IEEE 32nd International Conference on Distributed Computing Systems*, pp. 173–182. DOI: 10.1109/ICDCS.2012.30.

Hua, X. et al. (2022). "GenAD: General Unsupervised Anomaly Detection Using Multivariate Time Series for Large-Scale Wireless Base Stations". In: *Electronics Letters* 59.1. DOI: 10.1049/ell2.12683.

Huang, Ming et al. (2026). "HYDRA: A Multi-Level Hierarchy-Driven Approach for Robust Anomaly Detection in Time Series". In: *Proceedings of the ACM on Management of Data* 4.3, pp. 1–30. DOI: 10.1145/3802074.

Hyndman, R. J. and G. Athanasopoulos (2018). *Forecasting: Principles and Practice*. 2nd ed. Retrieved from https://otexts.com/fpp2/. OTexts.

— (2021). "Chapter 11 Forecasting hierarchical and grouped time series". In: *Forecasting: Principles and Practice*.

Keogh, Eamonn, Jessica Lin, and Ada Fu (2005). "HOT SAX: Efficiently Finding the Most Unusual Time Series Subsequence". In: *Fifth IEEE International Conference on Data Mining*, pp. 226–233. DOI: 10.1109/ICDM.2005.79.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

Kuhn, Harold W. (1955). "The Hungarian Method for the Assignment Problem". In: *Naval Research Logistics Quarterly* 2.1-2, pp. 83–97. DOI: 10.1002/nav.3800020109.

Law, Sean (2019). "STUMPY: A Powerful and Scalable Python Library for Time Series Data Mining". In: *Journal of Open Source Software* 4.39, p. 1504. DOI: 10.21105/joss.01504.

Lemaire, Pierre et al. (2024). "Early Detection of Critical Urban Events Using Mobile Phone Network Data". In: *PLOS ONE* 19.8, e0309093. DOI: 10.1371/journal.pone.0309093.

Lim, Bryan and Stefan Zohren (2021). "Time series forecasting with deep learning: A survey". In: *Philosophical Transactions of the Royal Society A* 379.2194, p. 20200209.

Linares-Barrera, Maria Luisa et al. (2024). "Evolutionary Feature Selection for Time-Series Forecasting". In: *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, pp. 395–399. DOI: 10.1145/3605098.3636191.

Lobo, Jorge M, Alberto Jiménez-Valverde, and Raimundo Real (2008). "AUC: a misleading measure of the performance of predictive distribution models". In: *Global ecology and Biogeography* 17.2, pp. 145–151.

Mefleh, A. et al. (2025). "Beyond the Norm: Unsupervised Anomaly Detection in Telecommunications with Mahalanobis Distance". In: *Computers* 14.12, p. 561. DOI: 10.3390/computers14120561.

Monge, Gaspard (1781). "Mémoire sur la théorie des déblais et des remblais". In: *Histoire de l'Académie Royale des Sciences*. Paris, pp. 666–704.

O'Neill, Michael and Conor Ryan (2001). "Grammatical Evolution". In: *IEEE Transactions on Evolutionary Computation* 5.4, pp. 349–358. DOI: 10.1109/4235.942529.

O'Neill, M. and C. Ryan (2003). *Grammatical Evolution: Evolutionary Automatic Programming in an Arbitrary Language*. Springer.

Oguz, H. T. and A. Kalaycioglu (2022). "Anomaly Detection in Multi-Tiered Cellular Networks Using LSTM and 1D CNN". In: *EURASIP Journal on Wireless Communications and Networking* 2022.1. DOI: 10.1186/s13638-022-02183-7.

Papadimitriou, C. H. and K. Steiglitz (1982). *Combinatorial optimization: Algorithms and complexity*. Prentice Hall.

Papadimitriou, Christos H and Kenneth Steiglitz (1998). *Combinatorial optimization: algorithms and complexity*. Courier Corporation.

Ruff, Lukas et al. (2021). "A unifying review of deep and shallow anomaly detection". In: *Proceedings of the IEEE* 109.5, pp. 756–795.

Sultan, Kashif, Hazrat Ali, and Zhang Zhang (2018). "Call Detail Records Driven Anomaly Detection and Traffic Prediction in Mobile Cellular Networks". In: *IEEE Access* 6, pp. 41728–41737. DOI: 10.1109/ACCESS.2018.2859756.

Vazirani, Vijay V (2001). *Approximation algorithms*. Vol. 1. Springer.

Wickramasuriya, Shanika L, George Athanasopoulos, Rob J Hyndman, et al. (2015). "Forecasting hierarchical and grouped time series through trace minimization". In: *Department of Econometrics and Business Statistics, Monash University* 105, pp. 6–1.

Wickramasuriya, Shanika L, George Athanasopoulos, and Rob J Hyndman (2019). "Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization". In: *Journal of the American Statistical Association* 114.526, pp. 804–819.

Wolff, Bar et al. (2022). "Composite Anomaly Detection via Hierarchical Dynamic Search". In: *2022 IEEE International Symposium on Information Theory*, pp. 2421–2426. DOI: 10.1109/ISIT50566.2022.9834631.

Xue, Bing et al. (2016). "A Survey on Evolutionary Computation Approaches to Feature Selection". In: *IEEE Transactions on Evolutionary Computation* 20.4, pp. 606–626. DOI: 10.1109/TEVC.2015.2504420.

Yeh, Chin-Chia Michael et al. (2017). "Time Series Joins, Motifs, Discords and Shapelets: A Unifying View That Exploits the Matrix Profile". In: *Data Mining and Knowledge Discovery* 32.1, pp. 83–123. DOI: 10.1007/s10618-017-0519-9.

Zhu, Yan et al. (2016). "Matrix Profile II: Exploiting a Novel Algorithm and GPUs to Break the One Hundred Million Barrier for Time Series Motifs and Joins". In: *2016 IEEE 16th International Conference on Data Mining*, pp. 739–748. DOI: 10.1109/ICDM.2016.0085.

Zlochin, Mark et al. (2004). "Model-based search for combinatorial optimization: A critical survey". In: *Annals of Operations Research* 131, pp. 373–395.